Comparaison de conteneurs pour un tas min

Benchmark expérimental en C++

ERRAMI Youssef

28 octobre 2026

Résultat principal. Pour ce tas binaire indexé, `std::vector` obtient les temps les plus faibles pour l'insertion, la recherche et la suppression du minimum. Les graphiques ci-dessous présentent les médianes de cinq essais pour chaque taille testée.

1 Présentation de mon programme

Dans ce projet, j'ai réalisé un programme C++ qui implémente un tas binaire min et compare son fonctionnement avec trois conteneurs : `std::vector`, `std::list` et `std::map`. Le menu permet de manipuler les trois tas en parallèle et de lancer un benchmark. Les opérations comparées sont l'insertion, la recherche, la suppression et l'affichage.

2 Structure du code

La logique du tas est définie une seule fois dans le modèle générique `MinHeap<Store>`. Le paramètre `Store` indique la manière dont les valeurs sont rangées. Trois classes adaptatrices, `VectorStore`, `ListStore` et `MapStore`, fournissent les mêmes opérations de base : accès par indice, ajout à la fin, retrait du dernier élément et accès à la taille. Ainsi, les trois variantes suivent le même algorithme ; seul le conteneur utilisé pour accéder aux valeurs change.

- `VectorStore` repose sur `std::vector`, qui accède directement à l'élément demandé par son indice.
- `ListStore` repose sur `std::list`. Pour atteindre l'indice demandé, il faut avancer dans les éléments de la liste.
- `MapStore` repose sur `std::map`, avec des indices successifs comme clés. L'accès à un indice s'effectue par recherche de cette clé dans l'arbre ordonné de la map.

3 Logique du tas min

Le tas est un arbre binaire complet représenté par un tableau logique. Sa racine, à l'indice zéro, contient la plus petite valeur. Pour un élément d'indice i, l'indice de son parent est $\lfloor (i-1)/2 \rfloor$ et ceux de ses enfants sont $2i+1$ et $2i+2$. La règle à maintenir après chaque modification est que la valeur d'un parent est inférieure ou égale à celles de ses enfants.

3.1 Insertion et réorganisation vers le haut

La fonction `insert` ajoute la nouvelle valeur à la fin du stockage. La fonction `siftUp` la compare ensuite à son parent. Si elle est plus petite, les deux valeurs sont échangées et la comparaison continue vers la racine. Le déplacement s'arrête dès que la propriété du tas est rétablie. Cette opération maintient la racine comme minimum, sans devoir trier toutes les valeurs.

3.2 Recherche

La fonction `contains` parcourt les éléments et s'arrête dès qu'elle trouve la valeur recherchée. Une recherche peut donc examiner une partie ou la totalité du tas. Le tas ordonne seulement les relations parent-enfants : il ne garantit pas que toutes les valeurs soient triées de gauche à droite. On ne peut donc pas appliquer à cette recherche les règles d'un arbre de recherche binaire.

3.3 Suppression et réorganisation

Pour supprimer une valeur donnée, `erase` la recherche dans le tas. Une fois l'indice trouvé, `eraseAt` remplace cette valeur par le dernier élément, puis retire la dernière case. Selon la valeur déplacée, le tas est réorganisé vers le haut avec `siftUp`, ou vers le bas avec `siftDown`. Pour supprimer le minimum, `removeMinimum` applique la même procédure à l'indice zéro. Après le remplacement de la racine, `siftDown` échange l'élément avec son plus petit enfant jusqu'au rétablissement de la propriété du tas.

3.4 Affichage

La méthode `toString` parcourt les indices du stockage et construit une chaîne contenant les valeurs dans leur ordre interne au tas. Cet ordre montre l'arbre dans sa représentation par tableau, mais ne correspond pas à un tri complet. L'option correspondante du menu écrit les trois chaînes dans `output.txt`.

4 Menu et fonctionnement du benchmark

Le menu conserve un tas distinct pour chaque conteneur. Lors d'une insertion, d'une recherche ou d'une suppression par valeur, la même valeur et la même opération sont appliquées aux trois structures. Le menu permet également de retirer le minimum, d'enregistrer l'affichage dans `output.txt`, de lancer le benchmark ou de quitter le programme.

Pour un benchmark, le programme génère une série de valeurs aléatoires puis construit chaque variante avec cette même série. Il mesure séparément le temps d'insertion, la recherche d'au plus 20 valeurs déjà insérées, la suppression successive des minimums jusqu'à vider le tas et la création de la chaîne d'affichage. Les mesures sont ensuite affichées dans la console et enregistrées dans `output.txt`.

Dans les résultats qui suivent, N désigne le nombre de valeurs insérées. Les tableaux et les graphiques présentent les médianes de cinq exécutions, en millisecondes.

5 Environnement de test

— Ordinateur : MSI Stealth GS66 12UGS.
— Processeur : Intel Core i7-12700H, 14 coeurs et 20 processeurs logiques (informations relevées dans Windows).
— Mémoire vive : 16 Go DDR5 (capacité annoncée et cohérente avec les 15,7 Gio de mémoire physique exposés par Windows).
— Carte graphique : NVIDIA GeForce RTX 3070 Ti Laptop GPU. Elle n'est pas sollicitée par ce benchmark, qui mesure du code C++ exécuté sur le processeur.
— Système : Windows 11 Pro.
— Compilateur : g++ 16.2.0 (MSYS2), avec les options `-std=c++17 -O2 -Wall -Wextra -pedantic`.

6 Résultats

6.1 Insertion de N valeurs

TABLE 1 – Temps médian d'insertion (ms)

N	`vector`	`list`	`map`
100	0,004 400	0,031 400	0,037 800
500	0,015 700	0,640 700	0,188 000
1000	0,029 100	2,938 900	0,367 700
2000	0,051 300	14,957 100	0,875 000
5000	0,125 200	128,559 500	2,044 300

6.2 Recherche de valeurs présentes

TABLE 2 – Temps médian de recherche (ms, jusqu'à 20 requêtes)

N	`vector`	`list`	`map`
100	0,001 000	0,022 500	0,014 700
500	0,002 200	1,231 700	0,132 500
1000	0,004 300	4,685 800	0,280 400
2000	0,008 000	25,391 200	0,894 800
5000	0,019 300	361,234 600	2,830 600

6.3 Suppression répétée du minimum

6.4 Préparation de l'affichage

7 Visualisation graphique

Les axes verticaux sont logarithmiques afin de rendre visibles les trois conteneurs malgré les écarts de temps importants. Chaque point représente la médiane de cinq essais. Les valeurs sont exprimées en millisecondes.

TABLE 3 – Temps médian pour vider le tas par suppressions du minimum (ms)

N	`vector`	`list`	`map`
100	0,007 200	0,039 400	0,057 400
500	0,034 000	1,214 800	0,455 600
1000	0,070 800	5,464 300	1,124 900
2000	0,159 000	29,574 400	3,522 300
5000	0,451 400	244,883 200	10,347 000

TABLE 4 – Temps médian de création de la chaîne d'affichage (ms)

N	`vector`	`list`	`map`
100	0,026 100	0,011 900	0,010 200
500	0,062 800	0,259 400	0,057 800
1000	0,069 200	0,966 400	0,096 600
2000	0,147 700	6,062 300	0,247 500
5000	0,305 800	50,326 700	0,583 100

7.1 Insertion

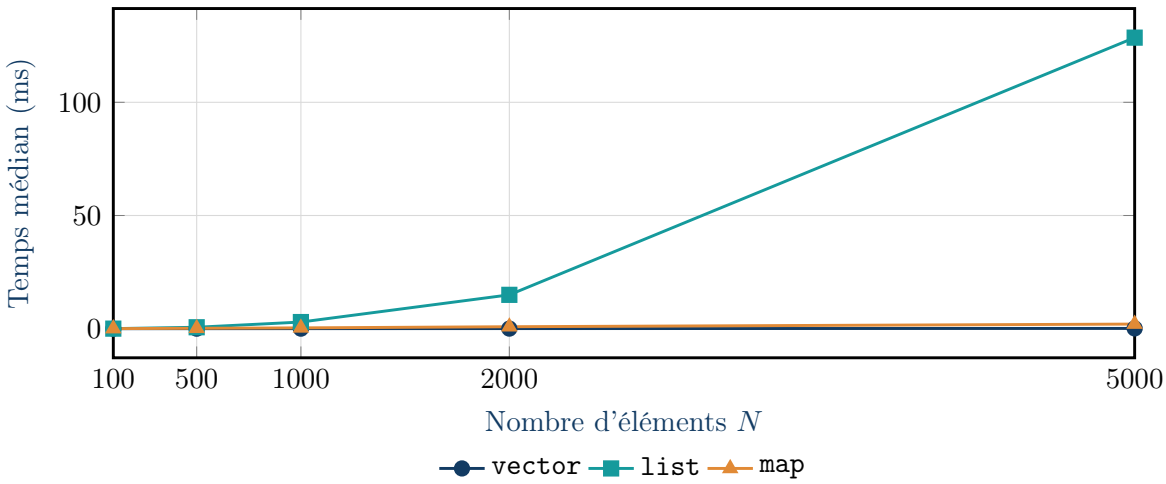


FIGURE 1 – Temps médian d'insertion de N éléments.

7.2 Recherche

7.3 Suppression répétée du minimum

7.4 Préparation de l'affichage

8 Analyse

Les résultats suivent directement la façon dont le tas utilise son stockage. Pour rétablir l'ordre, `siftUp` et `siftDown` accèdent à un élément, à son parent et à ses enfants par leurs indices. Avec `std::vector`, ces accès sont directs ; c'est pourquoi cette variante obtient les

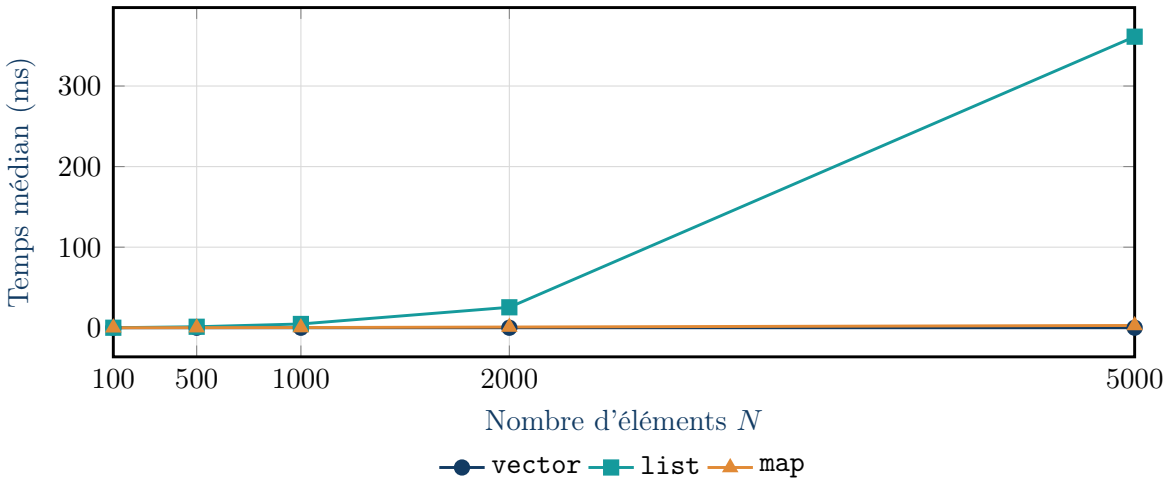


FIGURE 2 – Temps médian pour jusqu'à 20 recherches de valeurs présentes.

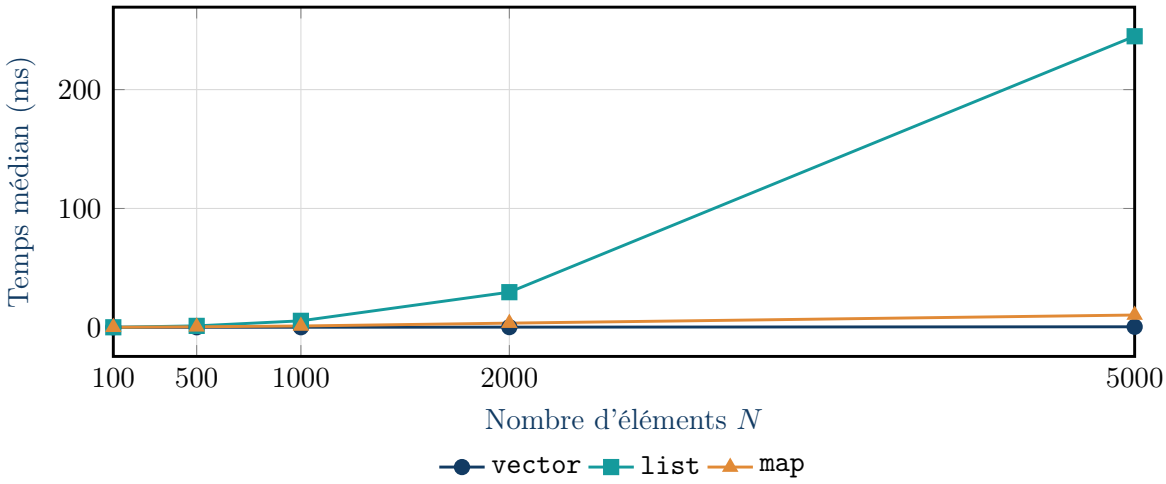


FIGURE 3 – Temps médian pour vider le tas en retirant successivement le minimum.

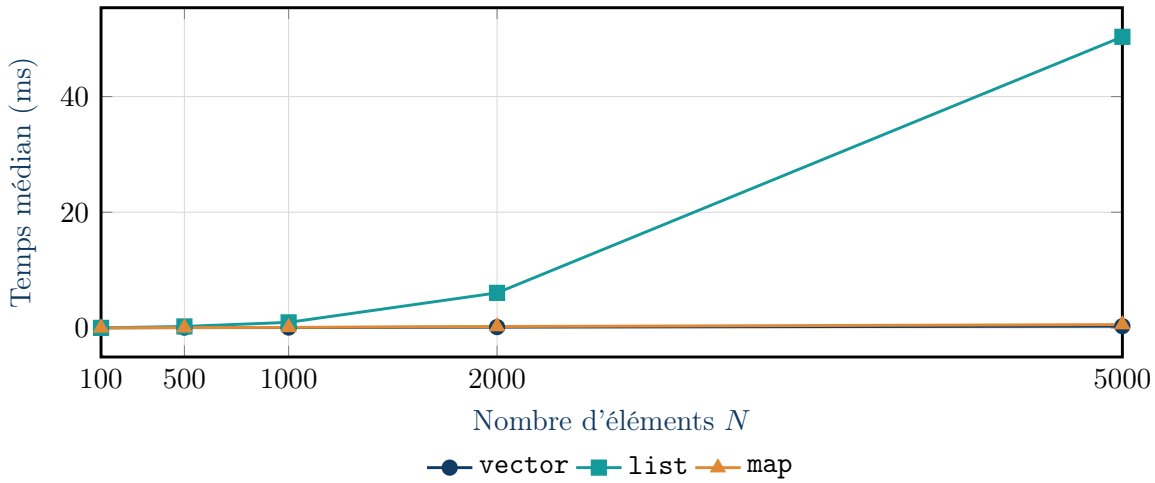


FIGURE 4 – Temps médian pour préparer la chaîne d'affichage du tas.

meilleurs temps pour l'insertion, la recherche et la suppression du minimum. Pour $N = 5000$, elle met 0,125 2 ms à insérer les valeurs, 0,019 3 ms à effectuer les recherches mesurées et 0,451 4 ms à vider le tas.

Dans `ListStore`, l'accès à un indice avance depuis le début de la liste. Comme les opérations du tas font de nombreux accès indexés, ce coût se répète pendant les échanges et les comparaisons. La recherche d'une valeur parcourt également le tas : avec $N = 5000$, les recherches mesurées sur `list` prennent 361,234 6 ms, contre 0,019 3 ms sur `vector`. La `map` est intermédiaire : ses clés permettent de retrouver un indice dans un arbre ordonné, mais chaque accès demande une recherche de clé au lieu d'un accès direct.

L'affichage illustre également le coût d'accès. La méthode `toString` parcourt les indices du tas ; elle bénéficie donc du vecteur, tandis que les accès répétés pénalisent la liste. Au total, le choix du conteneur ne modifie pas les règles du tas implémentées dans `MinHeap`, mais il modifie fortement le coût d'exécution de ces règles.

9 Conclusion

Ce projet m'a permis de mettre en pratique l'implémentation d'un tas min et de comparer l'effet du conteneur sur les opérations. Les résultats montrent que `std::vector` convient le mieux à cette logique : le tas calcule en permanence les indices des parents et des enfants, et le vecteur donne un accès direct à ces positions. La `std::map` peut conserver des éléments repérés par indice, mais chaque accès demande une recherche dans la structure. La `std::list` est la moins adaptée, car atteindre une position particulière demande de parcourir la liste. Le choix du conteneur doit donc correspondre à la manière dont l'algorithme accède aux données.